GRAPHICS PROGRAMMING – Mid-term Assignment

Source code

```
////////////////// SOLAR SYSTEM ///////////////////////
// sketch.js

var speed;

var sun_spin;

var earth_orbit_speed;
var earth_spin_speed;

var moon1_orbit_speed;
var moon2_orbit_speed;

var asteroid_orbit_speed;
// var moon1_spin_speed   -- always show same face to earth

function setup() {
    createCanvas(900, 900);
}

function draw() {
    get_speeds_and_background ()

    /// SOLAR SYSTEM ///
    sun_influence_area(sun_spin)

}

function sun_influence_area(rot_speed)
{
    push(); //1

        translate(width / 2, height / 2); // Initial translation to center of canvas
        sunRotation(rot_speed); // Sun spin speed

        // EARTH INFLUENCE AREA
        earth_influence_area(earth_spin_speed, earth_orbit_speed);

    pop() //1
}

function earth_influence_area(e_spin, e_orb) {
    push()

        // EARTH
        rotate(radians(e_orb)); // Earth orbit speed
        translate(0, 300);      // Earth Orbit radius

        earth(e_spin); // EARTH creation and spin speed
        shadow(80, 100) // EARTH shadow

        // MOONS INFLUENCE AREAS
        moon1_influence_area(e_spin, e_orb, moon1_orbit_speed)
        moon2_influence_area(e_spin, e_orb, moon2_orbit_speed)

    pop()
}

function moon1_influence_area(e_spin, e_orbit, orbit_speed) {
    push()
        // MOON 1

        spin_orbit_totals = orbit_speed-(e_spin + e_orbit)

        rotate(radians(spin_orbit_totals)); // Moon1 orbit speed
        translate(0, 100);                  // Moon1 orbit radius
```

```
        moon1(0) // Moon1 creation and adding spin speed
        moon_shadow_rotation(-spin_orbit_totals) // Moon1 shadow

        // ASTEROID
        asteroid_influence_area(spin_orbit_totals)
    pop()
}

function moon2_influence_area(e_spin, e_orbit, orbit_speed) {

    push()
        // MOON
        rotate(radians(orbit_speed-e_spin- e_orbit)); // Orbit
        translate(0, 50);
        moon2(0)
        moon_shadow_rotation(-orbit_speed+e_orbit+e_spin)

    pop()
}

function asteroid_influence_area(spin_orbit_totals)
{
    push()
        spin_orbit_totals = -spin_orbit_totals+asteroid_orbit_speed
        rotate(radians(spin_orbit_totals)); // Orbit
        translate(0, 30);       // Orbit radius
        celestialObj(color(255, 255, 0), 20); // ASTEROID

        asteroid_shadow(-asteroid_orbit_speed)
    pop()
}

/////////////// HELPER FUNCTIONS ////////////////////
function shadow(size, opacity)
{
    push()
        fill(0, 0, 0, opacity)
        arc(0, 0, size, size, 0, PI)
        stroke(200);
        // line(0, 0, 00, -50)
    pop()
}

function asteroid_shadow(totals)
{
    push()
        push()
        rotate(radians(0))
        marker_line() // Line poiniting to moon
        pop()

        rotate(radians(totals)); // Orbit
        shadow(30, 255)
    pop()
}

function moon_shadow_rotation(rot_speed)
{
    push()
        rotate(radians(rot_speed))
        shadow(30, 240)
    pop()
}

function moon1(rot_speed)
{
    push()
        rotate(radians(rot_speed)); // Spin
        marker_line() // Line poiniting to earth

        celestialObj(color("white"), 30);
    pop()
}
```

```
function moon2(rot_speed)
{
    push()
        rotate(radians(rot_speed)); // Spin
        marker_line() // Line poiniting to earth

        celestialObj(color("white"), 30);
    pop()
}

function marker_line()
{
    push()
        stroke("red")
        strokeWeight(2)
        line(0,0,0,-30)
    pop()
}


function earth(rot_speed){
    push()
        rotate(radians(rot_speed)); // Spin
        celestialObj(color("blue"), 80);
    pop()

}
function sunRotation(rot_speed) {
    push()  //2
        rotate(radians(rot_speed)); // Spin
        celestialObj(color(255, 150, 0), 200); // SUN
    pop()   //2
}

function celestialObj(c, size) {
    push()

        strokeWeight(5);
        fill(c);
        stroke(0);
        ellipse(0, 0, size, size);
        line(0, 0, size / 2, 0);
    pop();
}

function get_speeds_and_background()
{
    background(0);

    speed = frameCount;

    sun_spin = speed/3

    earth_orbit_speed = speed
    earth_spin_speed = speed

    moon1_orbit_speed = -2*speed
    moon2_orbit_speed =4.2*speed

    asteroid_orbit_speed = speed*2
}



///////////////// ASRTEROID GAME //////////////////
//##############################################################################################
//index.html

<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>asteroids games clone</title>
```

```
    <script src="libraries/p5.min.js" type="text/javascript"></script>
    <script src="libraries/p5.sound.min.js" type="text/javascript"></script>

    <script src="bulletSystem.js" type="text/javascript"></script>
    <script src="asteroidSystem.js" type="text/javascript"></script>
    <script src="spaceship.js" type="text/javascript"></script>
    <script src="sketch.js" type="text/javascript"></script>

    <script src="particle.js" type="text/javascript"></script>
    <script src="particleSmoke.js" type="text/javascript"></script>
    <script src="emitter.js" type="text/javascript"></script>
    <script src="emitterSmoke.js" type="text/javascript"></script>
    <script src="emitterExplosion.js" type="text/javascript"></script>
    <script src="particleExplosion.js" type="text/javascript"></script>
    <script src="particleBurn.js" type="text/javascript"></script>
    <script src="emitterBurn.js" type="text/javascript"></script>



    <style> body {padding: 0; margin: 0;} canvas {vertical-align: top;} </style>
  </head>
  <body>
  </body>
</html>



// sketch.js
var spaceship;
var asteroids;
var atmosphereLoc;
var atmosphereSize;
var earthLoc;
var earthSize;
var starLocs = [];
var running = true  // Indciaes if the game is running
var stratosphere = 1.15

/// I modified code very much. I created paricle systems and emitter for them. All of this codr is
created based on my previous experience. There may remain some of the original code however it is also
modified where it was necessary.

////////////////////////////////////////////////
function setup() {
  createCanvas(1200,800);

  spaceship = new Spaceship();
  asteroids = new AsteroidSystem();

  //location and size of earth and its atmosphere
  atmosphereLoc = new createVector(width/2, height*2.9);
  atmosphereSize = new createVector(width*3, width*3);
  earthLoc = new createVector(width/2, height*3.1);
  earthSize = new createVector(width*3, width*3);
}

////////////////////////////////////////////////
function draw() {
  background(0);
  sky();

  spaceship.run();
  asteroids.run();

  drawEarth();

  checkCollisions(spaceship, asteroids); // function that checks collision between various elements
}

////////////////////////////////////////////////
//draws earth and atmosphere
function drawEarth(){
  noStroke();
```

```
  //draw atmosphere
  fill(0,0,255, 50);
  ellipse(atmosphereLoc.x, atmosphereLoc.y, atmosphereSize.x,  atmosphereSize.y);

  fill(0,0,255, 35);
  ellipse(atmosphereLoc.x, atmosphereLoc.y, atmosphereSize.x*stratosphere,
 atmosphereSize.y*stratosphere);

  //draw earth
  fill(100,255);
  ellipse(earthLoc.x, earthLoc.y, earthSize.x, earthSize.y);
}

////////////////////////////////////////////////
//checks collisions between all types of bodies
function checkCollisions(spaceship, asteroids){

  //space ship-2-asteroid collisions
  for(var i=0;i<asteroids.locations.length;i++){
    if (isInside(spaceship.location,
                 spaceship.collisionRadius,
                 asteroids.locations[i],
                 asteroids.diams[i])){
      gameOver()
    }
  }

  //asteroid-2-earth collisions
  for(var i=0;i<asteroids.locations.length;i++){
    if (isInside(earthLoc,
                earthSize.x,
                asteroids.locations[i],
                asteroids.diams[i])) {
      gameOver()
    }
  }

  //spaceship-2-earth
  if (isInside(earthLoc,
              earthSize.x,
              spaceship.location,
              spaceship.collisionRadius)){
    gameOver()
  }

  //spaceship-2-atmosphere
  if (isInside(atmosphereLoc,
    atmosphereSize.x,
    spaceship.location,
    spaceship.collisionRadius)){
  spaceship.setNearEarth(earthLoc)
  }

  // asteroid-2-atmosphere
  for (var i=0; i<asteroids.diams.length;i++){
    if (isInside(asteroids.locations[i],
          asteroids.diams[i],
          atmosphereLoc,
          atmosphereSize.x*stratosphere,)
      ){
    asteroids.burn(i)
    }
  }
  //bullet collisions
  bulletNum = spaceship.bulletSys.getBulletsNum()
    for (var i=0; i<asteroids.diams.length;i++){
      for (var j=0; j<bulletNum;j++){
        if (asteroids.diams.length>0){ // Calls isInside function only when any asteroid exists
          if (isInside(asteroids.locations[i],asteroids.diams[i], spaceship.bulletSys.bullets[j],
spaceship.bulletSys.diam) ){

            asteroids.destroy(i,j,spaceship.bulletSys)
```

```
            i=0 // Restarts the the asteroids loop - asteroid was removed so the indices became
incorrect
          }
        }
      }
    }

}

////////////////////////////////////////////////
//helper function checking if there's collision between object A and object B
function isInside(locA, sizeA, locB, sizeB){

    if (dist(locA.x,locA.y, locB.x,locB.y) <sizeA/2+sizeB/2){  // IS INSIDE
      return true
    }
    else{       //IS OUTSIDE
      return false
    }
}


////////////////////////////////////////////////
// function that ends the game by stopping the loops and displaying "Game Over"
function gameOver(){
  fill(255);
  textSize(80);
  textAlign(CENTER);
  text("GAME OVER\n press ENTER to restart", width/2, height/2)
  running = false


  noLoop();

}

// Resets spaceship, bullets and asteroids. Reinitiates the draw function loop
function restartGame(){
  console.log("RESTARTING")
  running = true
  spaceship.reset()
  asteroids.reset()
  loop()
}

////////////////////////////////////////////////
// function that creates a star lit sky
function sky(){
  push();
  while (starLocs.length<300){
    starLocs.push(new createVector(random(width), random(height)));
  }
  fill(255);
  for (var i=0; i<starLocs.length; i++){
    rect(starLocs[i].x, starLocs[i].y,2,2);
  }

  if (random(1)<0.3) starLocs.splice(int(random(starLocs.length)),1);
  pop();
}

////////////////////////////////////////////////////////////////////////
//////////////////////////
function keyPressed(){
  // THRUSERT ON
  spaceship.keyPressed()

  if (keyIsPressed && keyCode === 32){ // if spacebar is pressed, fire!
    spaceship.fire();
  }
}

function keyReleased(){
  // THRUSTER OFF
```

```
  spaceship.keyReleased()

  // FIRE BULLET


  // RESET GAME WHEN GAME OVER
  if (keyCode == ENTER){
    // Restarts game when gameOver was called earlier
    if (running== false){
      restartGame()
    }
  }
}

//// asteroidSystem.js/////////
class AsteroidSystem {

  //creates arrays to store each asteroid's data
  constructor(){

    this.reset()
  }
  reset(){
    this.locations = [];
    this.velocities = [];
    this.accelerations = [];
    this.diams = [];
    this.explosions =[]
    this.runingExplosions = []
    this.burinigEmitter = []
    this.colours =[]
    this.score = 0

    this.numBurnParcicles = 0
    this.numExplosionParticles = 0

    this.isBurning = []

  }
  run(){
      this.spawn();
      this.move();
      this.draw();


  }

  // spawns asteroid at random intervals
  spawn(){
    if (random(1)<(0.005+0.0003*this.score)){
      this.accelerations.push(new createVector(0,random(0.1,1)));
      this.velocities.push(new createVector(0, 0));
      this.locations.push(new createVector(random(width), 0));
      this.diams.push(random(45,80));

      let astColor = [random(255),random(255),random(255)]
      this.colours.push(astColor)


      this.explosions.push(new EmitterExplosion())
      this.burinigEmitter.push(new EmitterBurn)

      this.isBurning.push(false)
    }
  }

  //moves all asteroids
  move(){
    for (var i=0; i<this.locations.length; i++){
      this.velocities[i].add(this.accelerations[i]);
      this.locations[i].add(this.velocities[i]);
      this.accelerations[i].mult(0);
    }
  }
```

```
applyForce(f){
  for (var i=0; i<this.locations.length; i++){
    this.accelerations[i].add(f);
  }
}

//draws all asteroids
draw(){
  noStroke();
  fill(200);
  for (var i=0; i<this.locations.length; i++){
    push()
    fill(this.colours[i][0]/2,this.colours[i][1]/2,this.colours[i][2]/2)
    ellipse(this.locations[i].x, this.locations[i].y, this.diams[i], this.diams[i]);
    pop()
  }

  for (var i = 0; i<this.runingExplosions.length; i++)
  {
    this.runingExplosions[i].run()
  }

  for  (var i=0; i<this.burinigEmitter.length; i++)
  {
    this.burinigEmitter[i].run()
  }

  // this.countBurningParicles()
  this.countExplosionParticles()
  this.coutnScore()
}

//function that calculates effect of gravity on each asteroid and accelerates it
calcGravity(centerOfMass){
  for (var i=0; i<this.locations.length; i++){
    var gravity = p5.Vector.sub(centerOfMass, this.locations[i]);
    gravity.normalize();
    gravity.mult(.001);
    this.applyForce(gravity);
  }
}

//destroys all data associated with each asteroid
destroy(index, bulletIndex,bulletSys){
  this.explosions[index].explode(
    this.locations[index].x,
    this.locations[index].y,
    this.velocities[index].x,
    this.velocities[index].y,
    this.colours[index],
    bulletIndex,
    bulletSys,
    this.isBurning[index],
    this.numExplosionParticles)
  this.runingExplosions.push(this.explosions[index])
  this.explosions.splice(index,1)

  this.locations.splice(index,1);
  this.velocities.splice(index,1);
  this.accelerations.splice(index,1);
  this.diams.splice(index,1);
  this.colours.splice(index,1);
  this.burinigEmitter.splice(index,1)
  this.isBurning.splice(index,1)

  this.score += 1
}


burn(index){
  this.isBurning[index] = true
  this.colours[index] = [255,50,50]
```

```
    if (frameCount%(2+int(this.numBurnParcicles/60))==0){ //reduce number of particles if there are
too many asteroids burning for performance
      this.buriniqEmitter[index].addParticle(this.locations[index].x+random(-this.diams[index]/2,
                                             this.diams[index]/2),
                                             this.locations[index].y,
                                             random(-2,2),0)}
  }
  countExplosionParticles(){ //counts number of explosion particles
    if (frameCount%5==0){
      let particles = 0
      this.runingExplosions.forEach(function(EmitterExplosion){
        particles += EmitterExplosion.particles.length
      })
      this.numExplosionParticles= particles

    }
  }

  countBurningParicles(){ //counts number of burning particles
    if (frameCount%5==0){
      let particles = 0
      this.buriniqEmitter.forEach(function(EmitterBurn){
        particles += EmitterBurn.particles.length
      })
      this.numBurnParcicles= particles

    }
  }

  coutnScore(){
    fill("orange")
    textSize(40)
    // change font to boxy one
    textFont ("Consolas")



    text("Score: "+this.score, 50, 50)
  }

}



/////////////// bulletSystem.js///////////////
class BulletSystem {

  constructor(){
    this.bullets = [];
    this.velocity = new createVector(0, -5);
    this.diam = 10;
  }

  run(){
      this.move();
      this.draw();
      this.edges();
  }

  fire(x, y){
    if (this.bullets.length < 5) // limit the number of bullets
    {
      this.bullets.push(createVector(x,y));
    }
  }

  //draws all bullets
  draw(){
    fill(255);
    for (var i=0; i<this.bullets.length; i++){
      ellipse(this.bullets[i].x, this.bullets[i].y, this.diam, this.diam);
    }
  }
```

```
  //updates the location of all bullets
  move(){
    for (var i=0; i<this.bullets.length; i++){
      this.bullets[i].y += this.velocity.y;
    }
  }

  //check if bullets leave the screen and remove them from the array
  edges(){
      for (var i = this.bullets.length-1;i>=0;i--){
        let bullet = this.bullets[i];
        if (bullet.y <0) {
          this.bullets = this.bullets.slice(1);
        }
      }

  }

  getBulletsNum(){
    return this.bullets.length
  }
}


//////////emitter.js//////////////

class Emitter{
  // Class based on the particle system introduced in the WEEK 3 lecture

  constructor(){
    this.particles = []
  }
  run(){
    this.draw()
  }
  draw(){
    for (var i=0; i< this.particles.length;i++){
      var paritcle = this.particles[i]

      this.particleGravity(paritcle)
      this.particleFriction(paritcle)


      // RUN PARTICLES
      paritcle.run();

      paritcle.age -=1
      this.removeOldParticle(paritcle)
    }
  }

  addParticle(locX,locY){
    this.particles.push(new Particle(locX,locY))

  }

  clearParticles(){
    this.particles =[]
  }

  removeOldParticle(particle){
    if ( this.particles.length>0 && particle.age <0){
      this.particles.shift()
    }
  }

  particleGravity(particle){
    // GRAVITY
    var gravity = createVector(0,0.0);
    particle.applyForce(gravity);
  }

  particleFriction(particle){
    // FIRCTION
```

```
    var friction = particle.velocity.copy()
    friction.mult(-1)
    friction.normalize();
    friction.mult(particle.friction_skalar + map(particle.age,300,0, 0,0.15));

    particle.applyForce(friction);
  }

  reset(){
    this.clearParticles()
  }



}



///////////////// emitterBurn.js //////////////////

class EmitterBurn extends Emitter{
    constructor(){
        super()


    }

    addParticle(locX,locY,velocityX,velocityY){ //OVERRIDE
        // Adds multiple particles
        for (var i=0; i< 2;i++){
            // Adds randomness to the location and velocity
            locX += random(-1,1)
            locY += 5
            velocityX += 0
            velocityY += 0

            this.particles.push(new ParticleBurn(locX,locY,velocityX,velocityY))
        }
    }


    burn(index){
        this.colours[index] = [255,50,50]
        this.burinigEmitter[index].addParticle(this.locations[index].x+random(-this.diams[index]/2,
this.diams[index]/2),this.locations[index].y)
    }

    stylePartcle(){  //OVERIDE
        noStroke()

        var alpha = map(this.age, this.maxAge,0,125,0);

        fill(this.r,this.g,this.b,alpha);


    }


}////////////// emitterExplosion.js //////////////

class EmitterExplosion extends Emitter{
    constructor(astColor){
        super()
    this.bulletVelocity= createVector(0,0)
    this.offset= createVector(0,0)
    this.astVelocity= createVector(0,0)
    this.explosionParitcles = []

    }
    run(){
        this.draw()
      }
```

```
    explode(locX,locY, velocityX,velocityY, astColor, j,bulletSys,burning,numExplodingParticles){
//OVERRIDE

        //// NEW VECTORS
        this.bulletVelocity = createVector(bulletSys.velocity.x, bulletSys.velocity.y) // bullet
velocity
        this.offset = createVector(bulletSys.bullets[j].x-locX, bulletSys.bullets[j].y-locY) // offset
vector  from bullet to asteroid
        this.astVelocity = createVector(velocityX, velocityY) // asteroid velocity

        // Add all vectors together
        let newParticleVelocit = p5.Vector.add(this.bulletVelocity, this.offset)
        newParticleVelocit = p5.Vector.add(newParticleVelocit , this.astVelocity)

        let randomMagn = random(1,5)
        let randomNum = random(100,300)
        let burnFactor = 1
        if (burning){
            burnFactor = 2
        }

        for (var i=0; i< randomNum*burnFactor/int(1+numExplodingParticles/150);i++){
            // Create new particle with velocity based on bullet and asteroid velocity and offset at
the time of collision
            var randVector = createVector(random(-5,5),random(-5,5))
            var randSkalar = random(0.01,1)
            let v = createVector(0,0)

            v.add(this.astVelocity)  // Adds asteroid velocity - neglible effect due to friciton
            v.add(randVector)

            v.add(p5.Vector.mult(newParticleVelocit,-randSkalar)) // Adds new partile vector - sum of
bullet speed, offset and asteroid speed

            if (i%6==0){ // Bullet trace effect when hit - some paricles fill go straight up
                v.y +=-abs(v.x)
                v.x =0
                v.rotate((random(-0.3,0.3)**1))
            }
            else if (i%2==0){ // Offset effect - some paricles will go in the direction indicated by
newParticleVelocity -- like bounced biliard ball
                v.rotate(random(-0.5,0.5))
                v.mult(0.3)
            }
            else { // Remaining particle will propagate in every direction

                v.rotate(random(-PI,PI))
                v.mult((random(0.1,0.5)**1/2))
            }
            v.mult(randomMagn)


            this.particles.push(new ParticleExplosion(locX,locY,v.x,v.y, astColor))

        }

    }

}


////////////// emitterSmoke.js ///////////////

class EmitterSmoke extends Emitter{
    constructor(){
        super()

    }

    addParticle(locX,locY,velocityX,velocityY){ //OVERRIDE
        // Adds multiple particles
        for (var i=0; i< 2;i++){
            // Adds randomness to the location and velocity
```

```
            locX += random(-3,3)
            locY += random(-3,3)
            velocityX += random(-1,1)
            velocityY += random(-0.5,0.5)

            this.particles.push(new ParticleSmoke(locX,locY,velocityX,velocityY))
        }


    }

    burn(index){
        this.colours[index] = [255,50,50]
        this.burinigEmitter[index].addParticle(this.locations[index].x+random(-this.diams[index]/2,
                                               this.diams[index]/2),
                                               this.locations[index].y)
      }




}



///// particle.js /////////

class Particle {

    constructor(locX,locY){
      this.location = new createVector(locX, locY);
      this.maxAge= 300
      this.age = 300

      this.size = random(10,25);

      this.velocity = new createVector(random(-3,3), random(-3,3));
      this.acceleration = new createVector(0, 0);

      this.friction_skalar = random(0.0015,0.035) // Adds randomness to the friction
     }

    run(){
      this.draw();
      this.move();
    }

    ////////////////////////////////////////////////
    draw(){
      push()
      this.size =  map(this.age,this.maxAge,0, 10,random(20,65));
      this.stylePartcle()
      ellipse(this.location.x, this.location.y, this.size, this.size);
      pop()
    }

    move(){
      this.velocity.add(this.acceleration);
      this.location.add(this.velocity);
      this.acceleration.mult(0); // Clears the acceleration to be ready for next frame
      this.velocity.limit(7)
    }



    applyForce(force){
      this.acceleration.add(force);
    }

    stylePartcle(){
      noStroke()
      var r =   map(this.age, this.maxAge,0,255,0 );
      var g =   map(this.age, this.maxAge,0,200,0);
      var b =   map(this.age, this.maxAge,0,255,100);
```

```
      var alpha = map(this.age, this.maxAge,0,125,0);

      fill(r,g,b,alpha);


    }

  }



  ////// /particleBurn.js//////

class ParticleBurn extends Particle{
  // Modifies the base class for the smoke particles

  constructor(locX,locY,velocityX,velocityY){
    super(locX,locY)
    this.velocity = createVector(velocityX,velocityY)

    this.maxAge = 90
    this.age = 90
    this.size = random(1,3);



    }
    draw(){
      push()
      this.size =  map(this.age,this.maxAge,0, 10,random(10,20));
      this.stylePartcle()
      ellipse(this.location.x, this.location.y, this.size, this.size);
      pop()
    }

    stylePartcle(){
      noStroke()
      var r =   map(this.age, this.maxAge,0,255,200);
      var g =   map(this.age, this.maxAge,0,125,0);
      var b =   map(this.age, this.maxAge,0,0,100);
      var alpha = map(this.age, this.maxAge,0,125,50);

      fill(r,g,b,alpha);


    }



}



//// particleExplosion.js //////

class ParticleExplosion extends Particle{
    constructor(locX,locY,velocityX,velocityY, astColor){
        super(locX,locY,velocityX,velocityY)
        this.velocity.x = velocityX
        this.velocity.y = velocityY
        this.maxAge = 60
        this.age = 60
        this.size = random(20,40);
        this.astColor = astColor
        this.r =   map(this.age, this.maxAge,0,this.astColor[0],0 );
        this.g =   map(this.age, this.maxAge,0,this.astColor[1],0);
        this.b =   map(this.age, this.maxAge,0,this.astColor[2],0);
    }
```

```
    stylePartcle(){  //OVERIDE
        noStroke()

        var alpha = map(this.age, this.maxAge,0,125,0);

        fill(this.r,this.g,this.b,alpha);


    }


}




///// particleSmoke.js
class ParticleSmoke extends Particle{
  // Modifies the base class for the smoke particles

  constructor(locX,locY,velocityX,velocityY){
    super(locX,locY)
    this.velocity = createVector(velocityX,velocityY)

    this.maxAge = 50
    this.age = 50
    this.size = random(3,6);

    }


}




//////// spaceship.js ///////////////

class Spaceship {

    constructor(){
      this.emitterSmoke = new EmitterSmoke();  // EmitterSmoke is a subclass of Emitter - it is
responsible for drawing the particles from the thrusters
      this.reset() // Sets the spaceship to its initial state

    }

    reset(){
      // This function resets the spaceship to its initial state

      this.velocity = new createVector(0, 0);
      this.location = new createVector(width/2, height/2);
      this.acceleration = new createVector(0, 0);
      this.maxVelocity = 5;
      this.bulletSys = new BulletSystem();
      this.size = 50;
      this.collisionRadius = this.size/2

      this.thrusterRight = false;
      this.thrusterLeft = false;
      this.thrusterUp = false;
      this.thrusterDown = false;

      this.emitterSmoke.reset()
    }


    run(){
      // Runs the spaceship
      this.bulletSys.run();
      this.draw();
```

```
  this.move();
  this.edges();
  this.interaction();
  this.drawThrusters();
  this.emitterSmoke.run()
}

draw(){
  fill(125);
  triangle(this.location.x - this.size/2, this.location.y + this.size/2,
      this.location.x + this.size/2, this.location.y + this.size/2,
      this.location.x, this.location.y - this.size/2);

  /// FOR TESTING ONLY !!
  this.shipCollisionCircle()
}

move(){
    this.velocity.add(this.acceleration);
    this.velocity.limit(this.maxVelocity);
    this.location.add(this.velocity);

    this.acceleration.mult(0);
}

applyForce(f){
  this.acceleration.add(f);
}

interaction(){
    if (keyIsDown(LEFT_ARROW)){
      this.applyForce(createVector(-0.1, 0));
    }
    if (keyIsDown(RIGHT_ARROW)){
      this.applyForce(createVector(0.1, 0));
    }
    if (keyIsDown(UP_ARROW)){
      this.applyForce(createVector(0, -0.1));
    }
    if (keyIsDown(DOWN_ARROW)){
      this.applyForce(createVector(0, 0.1));
    }
}

fire(){
  this.bulletSys.fire(this.location.x, this.location.y);
}

edges(){
  if (this.location.x<0) {
    this.velocity.x *= random(-0.8,-1);
    this.location.x = 3;
    this.velocity.y += random(-2,3);
    }
  else if (this.location.x>width) {
      this.location.x = width-3;
      this.velocity.x *= random(-0.8,-1);
      this.velocity.y += random(-2,3);
    }
  else if (this.location.y<0) this.location.y = height;
  else if (this.location.y>height) this.location.y = 0;
  // if (this.location.x<0) this.location.x=width;
  // else if (this.location.x>width) this.location.x = 0;
  // else if (this.location.y<0) this.location.y = height;
  // else if (this.location.y>height) this.location.y = 0;
}

setNearEarth(earthLoc){
  // var gravity= createVector(0,0.05)
  var gravity = p5.Vector.sub(earthLoc, this.location).normalize().mult(0.05)
  var dir = gravity.copy().mult(20)  // dir is used to draw gravity vector

  var airResistance = this.velocity.copy().mult(-1/30)
```

```
  this.applyForce(gravity)
  this.applyForce(airResistance)

  // DRAW FORCE VECTORS
  this.drawGravityVector(dir)
  this.drawAirResistanceVector(airResistance)
}

drawThrusters(){
  // This function draws the thrusters - it uses the particle system to draw it
  if (this.thrusterRight==true){
      this.emitterSmoke.addParticle(this.location.x,this.location.y, this.velocity.x-10,0);
    }
    if (this.thrusterLeft==true){
      this.emitterSmoke.addParticle(this.location.x,this.location.y, this.velocity.x+10,0);
    }
    if (this.thrusterUp==true){
      this.emitterSmoke.addParticle(this.location.x,this.location.y, 0,this.velocity.y+10);
    }
    if (this.thrusterDown==true){
      this.emitterSmoke.addParticle(this.location.x,this.location.y, 0,this.velocity.y-10);
    }
}

keyPressed(){
  if (keyCode === RIGHT_ARROW){
    this.thrusterRight = true;
  }
  if (keyCode === LEFT_ARROW){
    this.thrusterLeft = true;
  }
  if (keyCode === UP_ARROW){
    this.thrusterUp = true;
  }
  if (keyCode === DOWN_ARROW){
    this.thrusterDown = true;
  }
}

keyReleased(){
  if (keyCode === RIGHT_ARROW){
    this.thrusterRight = false;
  }
  if (keyCode === LEFT_ARROW){
    this.thrusterLeft = false;
  }
  if (keyCode === UP_ARROW){
    this.thrusterUp = false;
  }
  if (keyCode === DOWN_ARROW){
    this.thrusterDown = false;
  }

}


//// MY FUNCITONS FOR DRAWING VECTORS ////
drawGravityVector(dir){
  push()
  stroke("yellow")
  strokeWeight(4)
  translate(this.location.x,this.location.y)
  line(0,0, dir.x*30,dir.y*30)
  pop()
}

drawAirResistanceVector(airResistance){
  push()
  stroke("red")
  strokeWeight(4)
  translate(this.location.x, this.location.y)
  line(0,0,airResistance.x*300,airResistance.y*300)
  pop()
}
```

```
    // NEW METHODS



    /// MY TESTERS !!!////
    shipCollisionCircle()
    {
      push()
      noFill()
      strokeWeight(2)
      stroke("red")
      ellipse(this.location.x,this.location.y,this.collisionRadius,this.collisionRadius)
      pop()
    }
  }

//####################################
//####################################
//####################################

////////// ANGRY BIRDS CLONE //////////
//sketch.js
// Example is based on examples from: http://brm.io/matter-js/, https://github.com/shiffman/p5-matter
// add also Benedict Gross credit


// MY CODE - original code was modified and spread into different functions. Except that original code
in the helper area at the bottom, all code is mine
var Engine = Matter.Engine;
var Render = Matter.Render;
var World = Matter.World;
var Bodies = Matter.Bodies;
var Body = Matter.Body;
var Constraint = Matter.Constraint;
var Mouse = Matter.Mouse;
var MouseConstraint = Matter.MouseConstraint;

var engine;
var propeller;
var boxes;
var birds;
var colors;
var ground;
var slingshotBird, slingshotConstraint;
var angle;
var angleSpeed;
var canvas;

// MY GLOBAL VARIABLES
var maxAngleSpeed = 0.4;
var birdSize = 20;
var boxSize;
var slingshotBirdInitPos;
var Events = Matter.Events
var score;
var towerHeight;
var towerWidth;


var ifGameOver;;
//////////////////////////////////////////////////////////
function setup() {
  canvas = createCanvas(1000, 600);

  birdBoxCollisionDetector = new birdBoxCollisionDetector();
  initializeGame();
}

//////////////////////////////////////////////////////////
function draw() {
```

```
  Engine.update(engine);

  background(0);

  drawGround();

  drawPropeller();

  drawTower();

  drawBirds();

  drawSlingshot();

  drawScore();

  checkForGameOver();


}
//////////////////////////////////////////////////////////
function initializeGame(){
  boxes = [];
  birds = [];
  colors = [];
  score = -3;
  angle=0;
  angleSpeed=0;
  boxSize= random(50,80)
  towerHeight = random(6,8);
  towerWidth = random(3,5);

  engine = Engine.create();  // create an engine

  setupGround();

  setupPropeller();

  setupTower();

  setupSlingshot();

  setupMouseInteraction();

  birdBoxCollisionDetector.setup();
}

//use arrow keys to control propeller
function keyPressed(){
  if (keyCode == ENTER){
    restartGame();

  }
  if (keyCode == LEFT_ARROW){
    // Decrease angle speed and go counetr-clockwise
    if (angleSpeed > -maxAngleSpeed){
      angleSpeed -= 0.04;
    }
  }
  else if (keyCode == RIGHT_ARROW){
    // Increase angle speed and go clockwise
    if (angleSpeed < maxAngleSpeed){
      angleSpeed += 0.04;
    }
  }
}
//////////////////////////////////////////////////////////
function keyTyped(){
  //if 'b' create a new bird to use with propeller
  if (key==='b'){
    setupBird();
  }

  //if 'r' reset the slingshot
```

```
  if (key==='r'){
    removeFromWorld(slingshotBird);
    removeFromWorld(slingshotConstraint);
    setupSlingshot();
  }
}

function restartGame(){
  initializeGame()


}

////////////////////// MY CODE ////////////////////////
function setupPropeller(){
  // creates a propeller
  propeller = Bodies.rectangle(150, 480, 200, 15, {isStatic: true, angle: angle});

  // adds propeller to the world
World.add(engine.world, [propeller]);
}

function drawPropeller(){
  // update angle
  angle += angleSpeed;

  //set angle to the propeller
  Body.setAngle(propeller, angle);

  // assign anlgeSpeed to the propeller
  Body.setAngularVelocity(propeller, angleSpeed);

  drawVertices(propeller.vertices);
}

function setupBird(){
  // creates a bird
  let bird = Bodies.polygon(mouseX, mouseY, 1, birdSize,{restitution:0.8, friction:0.5,
label:"bird"});
  Body.setMass(bird,1)

  // keeps track of the bird object
  birds.push(bird);

  // adds bird to the world
  World.add(engine.world, [bird]);
}

function drawBirds(){
  // iterate through the birds array to draw individual birds
  for (let i=0; i<birds.length; i++){
    push()
    fill("red")
    drawVertices(birds[i].vertices);
    pop()

    // remove birds that went off screen
    if (isOffScreen(birds[i])){
      removeFromWorld(birds[i]);
      birds.splice(i,1);
      i--;
      score++;
    }
  }
}



function setupTower(){
  // Creating tower elements
  for (let i=0;i<towerHeight;i++){    // tower height in boxes
    for (let j=0;j<towerWidth;j++){  // tower width in boxes

      // Creatng single box
```

```
      let box = Bodies.rectangle(width*0.95-j*boxSize, 620-i*boxSize, boxSize,boxSize,
{label:"towerBox"})
      boxes.push(box);
      World.add(engine.world,[box])

      // Generatig random colour for the box
      boxColor = color(50,random(50,200),50)
      colors.push(boxColor)
    }
  }
}

function drawTower(){
  // Loops through the individual boxes of the tower and draws them
  for (let i=0;i<boxes.length;i++){
    push();

    fill(color(colors[i]));

    drawVertices(boxes[i].vertices);
    pop();

    // any box off screen is deleted
    if (isOffScreen(boxes[i])){
      removeFromWorld(boxes[i])
      boxes.splice(i,1);
      colors.splice(i,1)
      i--
      score++
    }
  }
}


function setupSlingshot(){
  // Creating vector containing slingshotBird inital position
  var slingshotBirdInitPos = new createVector(200, 200);

  // Renames for simplification
  var sBirdiPos = slingshotBirdInitPos;
  slingshotBird = Bodies.circle(sBirdiPos.x,sBirdiPos.y,30, {friction:0, restitution:0.95,
label:"slingshotBird"})
  Body.setMass(slingshotBird,10)
  World.add(engine.world, [slingshotBird])

  // Creating a contraint that is fixed to the world and to the slingshotBird - constraint is
destroyed when mouse released after dragging the slingshotBird (implemented in mouseReleased function)
  slingshotConstraint = Constraint.create({
    pointA:{x:sBirdiPos.x, y:sBirdiPos.y},
    bodyB: slingshotBird,
    pointB:{x:0,y:0},
    stiffness: 0.01,
    damping: 0.0001
  });
  World.add(engine.world, [slingshotConstraint])

}

function drawSlingshot(){
  // Draws Slingshot Bird
  push()
  fill("yellow")
  drawVertices(slingshotBird.vertices)
  pop()

  // Draw slingshot constraint
  drawConstraint(slingshotConstraint)
}


function drawScore(){
  // Shows how many boxes remain do be destroyed and total number of destroyed boxes (the score)
  push()
```

```
  // TO DESTROY
  textSize(26);
  fill("red");
  text("To destroy: " + boxes.length, 10, 30);

  // SCORE
  textSize(32);
  fill("yelow");
  text("Score: " + score, 10, 60);

  // DUST, SMALLBOX, TOWERBOX REMAINING
  textSize(20);
  fill(125);
  // show only boxes which label is "smallBox"
  let towerBoxes = boxes.filter(box => box.label == "towerBox");
  let smallBoxes = boxes.filter(box => box.label == "smallBox");
  let dusts = boxes.filter(box => box.label == "dust");
  text("REMAINING: TowerBoxes: " +towerBoxes.length + ", SmallBoxes: "+smallBoxes.length + ", Dust:
"+dusts.length,270, 30);
  pop()
}

function checkForGameOver(){
  // noloop if no boxes left
  if (boxes.length == 0){
    // display game over message
    push()
    textSize(70);
    fill('orange');
    text("GAME OVER", width/2-150, height/2);

    // display the final score
    textSize(30);
    fill('green');
    text("Boxes destroyed: " + score, width/2-150, height/2+50);
    fill(125);
    textSize(20);
    text("Press ENTER to restart", width/2-150, height/2+100)
    pop()

    // stop the game
    ifGameOver = true;
  }

  if (ifGameOver == false){
    drawScore();
  }
}

class birdBoxCollisionDetector{
  constructor(){
    self = this; // self has to be used as the Events on has its own this
  }

  setup(){
    Events.on(engine, 'collisionStart', function(event) {
      let pairs = event.pairs;

      // Loop over the pairs of objects that collided
      for (let i = 0; i < pairs.length; i++) {
        let bodyA = pairs[i].bodyA;
        let bodyB = pairs[i].bodyB;

        // Check if bodyA should be split
        if (self.shouldSplit(bodyA,bodyB)) {

          // Remove the original body from the world
          Matter.World.remove(engine.world, bodyA);
          // Storing the color of the destroyed box
          let index = boxes.indexOf(bodyA)
          let destroyedBoxColor = color("yellow")

          // Color is stored if body is in the boxex array
          if (index != -1){
```

```
          destroyedBoxColor = color(colors[index])
        }
        else{
          console.log("error - no color found")
        }

        self.removeFromBoxes(bodyA)

        score++;
        if (bodyA.label != "dust"){
          self.replaceWithStack(bodyA, destroyedBoxColor)
        };
      };
    };
  });
}

checkSpeedThreshold(bodyA,bodyB,minSpd){
  /// Check if the collision speed is above a threshold to generate split
  let collisionSpeedThreshold = minSpd;
  let relativeVelocity = Matter.Vector.sub(bodyA.velocity, bodyB.velocity);
  let collisionSpeed = Matter.Vector.magnitude(relativeVelocity);
  return collisionSpeed >= collisionSpeedThreshold;
}

shouldSplit(bodyA,bodyB){
  /// Check if body hit by a bird should split
  if (bodyB.label == "slingshotBird" || bodyB.label == "bird"){
    if (bodyA.label =="towerBox"){ // tower box hit by bird
      if (self.checkSpeedThreshold(bodyA,bodyB,15)){
        return true;
      }
    }
    /// Chcks if small box hit by bird should split
    else if(bodyA.label =="smallBox"){ // small box hit by bird
      if (self.checkSpeedThreshold(bodyA,bodyB,10)){
        return true;
      }
    }
    else if(bodyA.label =="dust"){ // dust hit by bird
      if (self.checkSpeedThreshold(bodyA,bodyB,5)){
        return true;
      }
    }
  }

  else{
    return false;
  }
}

removeFromBoxes(body) {
  let index = boxes.indexOf(body); // gets the index of the box in the boxes array

  if (index !== -1) { // box exists in the boxes array
    boxes.splice(index, 1); // removes box from the boxes array
    colors.splice(index, 1); // removes colour from the colours array
  }

  return index;
}

replaceWithStack(originalBox, destroyedBoxColor) {
  // Get the position and velocity of the original box
  let position = originalBox.position;
  let velocity = originalBox.velocity;
  let angularVelocity = originalBox.angularVelocity;

  // Remove the original box from the world
  let newColor  = color("gray")

  if (destroyedBoxColor){
    newColor = destroyedBoxColor;
  }
```

```
    Matter.World.remove(engine.world, originalBox);
    self.removeFromBoxes(originalBox);

    // Create a stack of rectangles

    let stackWidth = 3; // Width of the stack (number of rectangles)
    let stackHeight = 3; // Height of the stack (number of rectangles)

    let stackSize = boxSize / stackWidth; // Size of each rectangle in the stack
    if (originalBox.label == "smallBox"){
      stackSize = stackSize/3;
    }
    let stack = [];
    for (let i = 0; i < stackHeight; i++) {
      for (let j = 0; j < stackWidth; j++) {
        let x = position.x + j * stackSize;
        let y = position.y + i * stackSize;
        let label  = "smallBox"
        if (originalBox.label == "smallBox"){
          label = "dust"
        }
        let rectangle = Bodies.rectangle(x, y, stackSize, stackSize,{label:label});
        stack.push(rectangle);
      }
    }

    // Add the stack to the world
    Matter.World.add(engine.world, stack);

    // Adjust the position and velocity of each rectangle in the stack
    stack.forEach((rectangle, index) => {
      let x = position.x + ((index % stackWidth) - 0.5) * stackSize;
      let y = position.y + (Math.floor(index / stackWidth) - 0.5) * stackSize;
      Body.setPosition(rectangle, { x, y });
      Body.setVelocity(rectangle, velocity);
      Body.setAngularVelocity(rectangle, angularVelocity);
    });

    // Update the boxes and colors arrays with the new rectangles

    boxes = boxes.concat(stack);

    // Assigning unique collor for each stack of "dust" boxes
    // let newColor = color(random(100,150),random(100,150),20);
    if (originalBox.label == "smallBox"){
      newColor = color(random(100,200), 50,70)
    }

    colors = colors.concat(Array(stack.length).fill(newColor));



  }
}

//*********************************************************************
//  HELPER FUNCTIONS - DO NOT WRITE BELOW THIS line
//*********************************************************************

//if mouse is released destroy slingshot constraint so that
//slingshot bird can fly off
function mouseReleased(){
  setTimeout(() => {
    slingshotConstraint.bodyB = null;
    slingshotConstraint.pointA = { x: 0, y: 0 };
  }, 100);
}
////////////////////////////////////////////////////////
//tells you if a body is off-screen
function isOffScreen(body){
  var pos = body.position;
  return (pos.y > height || pos.x<0 || pos.x>width);
}
```

```
////////////////////////////////////////////////////////////
//removes a body from the physics world
function removeFromWorld(body) {
  World.remove(engine.world, body);
}
////////////////////////////////////////////////////////////
function drawVertices(vertices) {
  beginShape();
  for (var i = 0; i < vertices.length; i++) {
    vertex(vertices[i].x, vertices[i].y);
  }
  endShape(CLOSE);
}
////////////////////////////////////////////////////////////
function drawConstraint(constraint) {
  push();
  var offsetA = constraint.pointA;
  var posA = {x:0, y:0};
  if (constraint.bodyA) {
    posA = constraint.bodyA.position;
  }
  var offsetB = constraint.pointB;
  var posB = {x:0, y:0};
  if (constraint.bodyB) {
    posB = constraint.bodyB.position;
  }
  strokeWeight(5);
  stroke(255);
  line(
    posA.x + offsetA.x,
    posA.y + offsetA.y,
    posB.x + offsetB.x,
    posB.y + offsetB.y
  );
  pop();
}
```